

AEL-1

Md. Atiqur Rahman
Data Structures - 1

Data Structures

A data structure is a way of organizing and storing data in a computer so that it can be accessed and used efficiently.

Types of data structures,

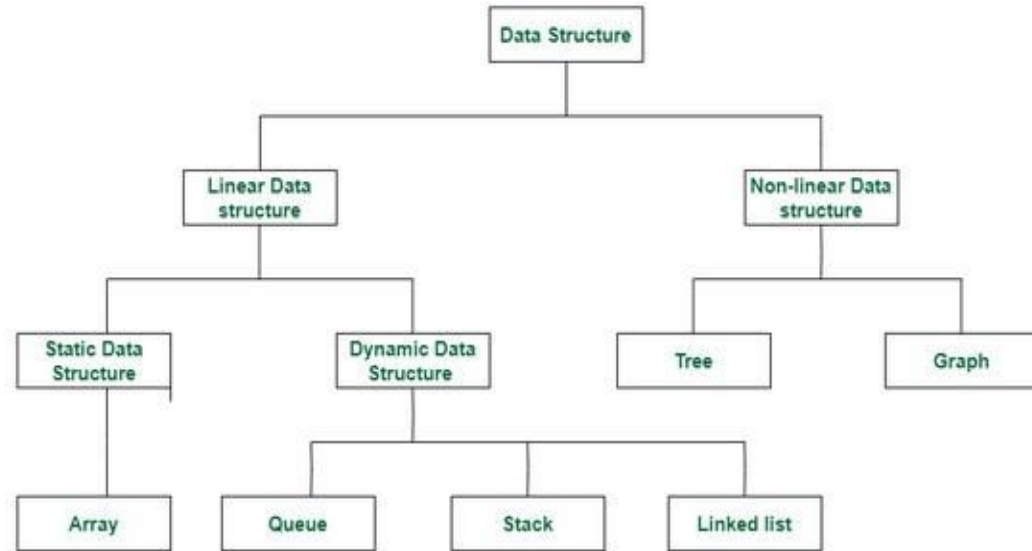
- Linear

- Static
 - Array
- Dynamic
 - Queue
 - Stack
 - Linked List

- Non-linear

- Tree (Heap)
- Graph

Classification of Data Structure



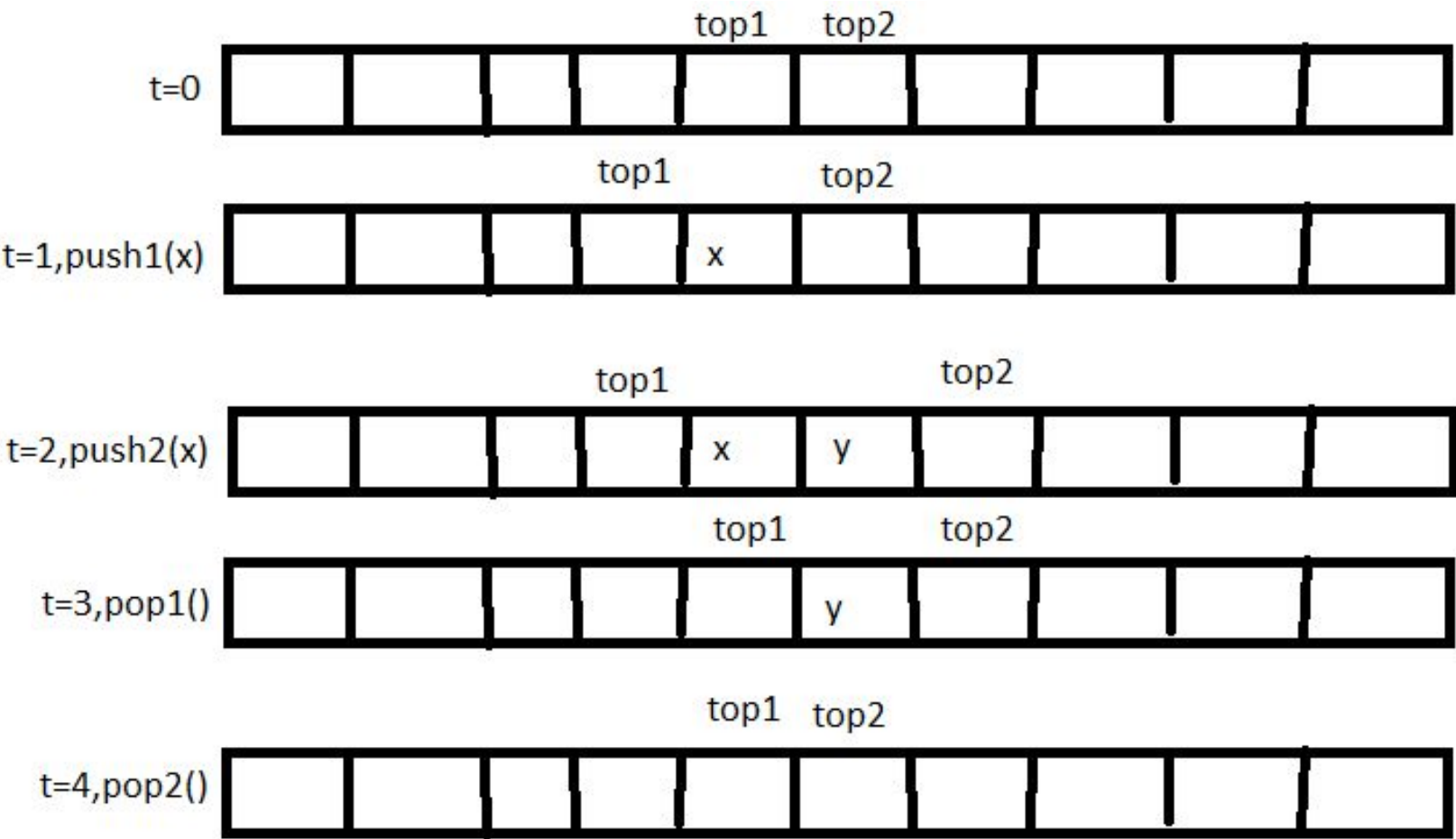
Definitions

- Linear : data elements are arranged sequentially.
- Non-linear : We can not traverse elements in a single run only.
- Static : has fixed memory size and easier to access the elements in a static data.
- Dynamic : Memory size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (space) complexity of the code.
- **Array** : An array is a collection of items of same data type stored at contiguous memory locations.
- **Queue** : A Queue is defined as a linear data structure which works almost like list in which all additions to the list are made at one end, and all deletions from the list are made at the other end, like FIFO (First In First Out) order.
- **Stack** : Stack is a linear data structure that follows a particular order LIFO which implies that the element that is inserted last, comes out first.
- **Linked List** : Linked List is a linear data structure, in which elements are not stored at a contiguous location, rather they are linked using pointers. Linked List forms a series of connected nodes, where each node stores the data and the address of the next node.

Implement two Stacks in an Array

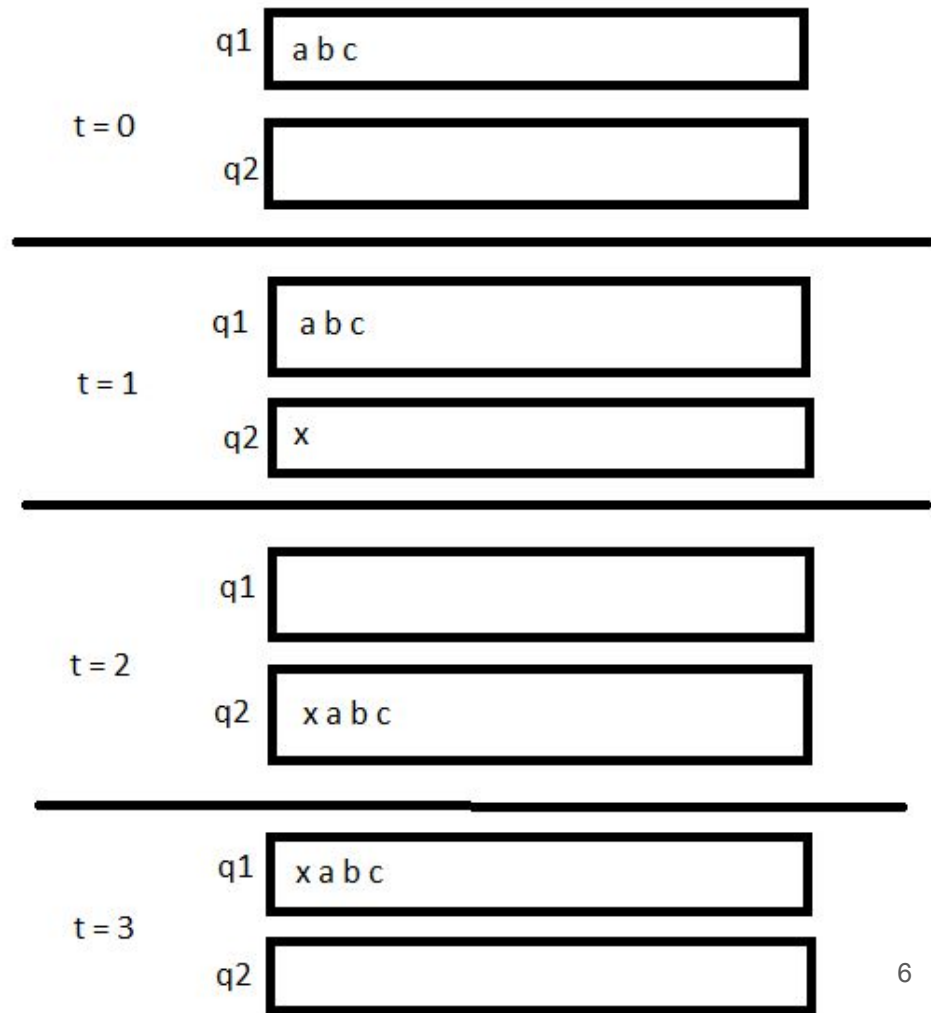
- To implement **push1()**:
 - First, check whether the **top1** is greater than 0
 - If it is then add an element at the top1 index and decrement top1 by 1
 - Else return Stack Overflow.
- To implement **push2()**:
 - First, check whether **top2** is less than $n - 1$
 - If it is then add an element at the top2 index and increment the top2 by 1
 - Else return Stack Overflow.
- To implement **pop1()**:
 - First, check whether the **top1** is less than or equal to $n / 2$
 - If it is then increment the top1 by 1 and return that element.
 - Else return Stack Underflow.
- To implement **pop2()**:
 - First, check whether the **top2** is greater than or equal to $(n + 1) / 2$
 - If it is then decrement the top2 by 1 and return that element.
 - Else return Stack Underflow.

Continued



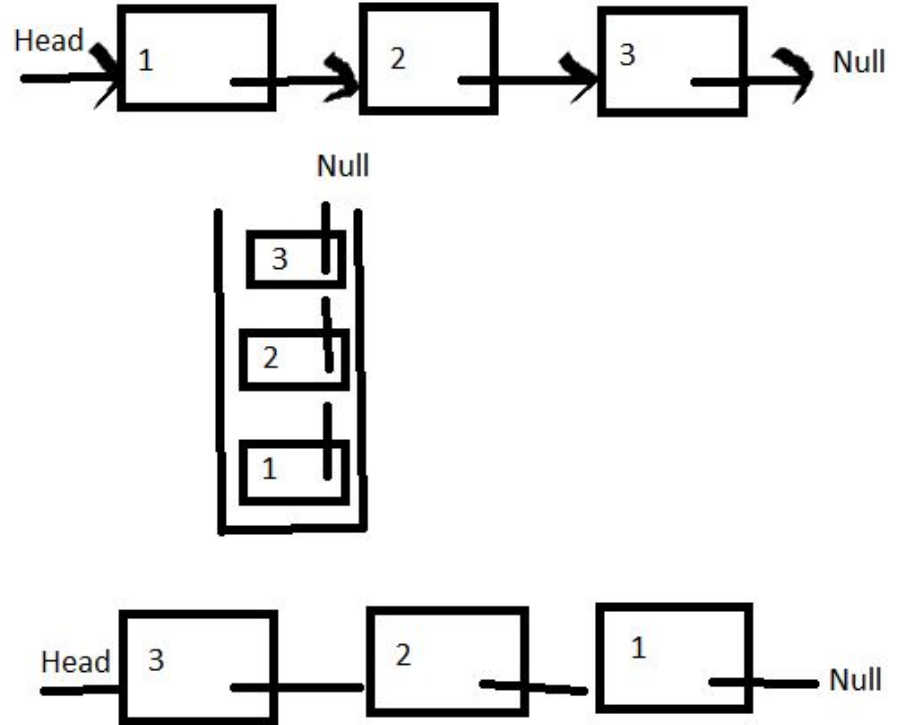
Implement Stack using queues

- Follow the below steps to implement the **push(s, x)** operation:
 - Enqueue x to q2.
 - One by one dequeue everything from q1 and enqueue to q2.
 - Swap the queues of q1 and q2.
- Follow the below steps to implement the **pop(s)** operation:
 - Dequeue an item from q1 and return it.



Reverse a Link List using Stack

- Store the nodes(values and address) in the stack until all the values are entered.
- Once all entries are done, Update the Head pointer to the last location(i.e the last value).
- Start popping the nodes(value and address) and store them in the same order until the stack is empty.
- Update the next pointer of last Node in the stack by NULL.



Most used built-in data structures

With the evolution of programming languages, data structures are now often used through abstractions, so we don't usually worry about their internal implementation. Here, we will take a closer look at how some commonly used structures actually work under the hood:

- Python's Dict

Python's Dict

- Python's dictionary is implemented using a Hash Table.
- $\text{index} = \text{hash}(\text{key}) \% \text{table_size}$
- If collision occurs, it used Perturbation-based probing. This sort of probing avoids clustering better than simple linear probing.

$i = \text{hash} \% \text{size}$

while slot occupied:

$i = (i * 5 + 1 + \text{perturb}) \% \text{size}$

$\text{perturb} \ll= 5$

- Worst case ($O(n)$) happens because, hash collisions and adversarial input .

Operation	Average	Worst
Insert	$O(1)$	$O(n)$
Lookup	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

Insertion and Lookup

```
def insert(table, key, value):
```

```
    h = hash(key)
```

```
    i = h % table.size
```

```
    while table[i] is occupied:
```

```
        if table[i].key == key:
```

```
            table[i].value = value
```

```
            return
```

```
        i = probe_next(i)
```

```
    table[i] = (key, value, h)
```

```
def lookup(table, key):
```

```
    h = hash(key)
```

```
    i = h % table.size
```

```
    while table[i] is not empty:
```

```
        if table[i].key == key:
```

```
            return table[i].value
```

```
        i = probe_next(i)
```

```
    return KeyError
```

Deletion

Python dict uses a Hash Table with open addressing, so elements are part of a probe sequence. If you remove an entry completely, you may break that sequence.

Suppose keys collided and were placed like this:

Index: 3 4 5

[A] [B] [C]

Now if you delete B and make it empty:

Index: 3 4 5

[A] [] [C]

Searching for C:

- Starts at index 3
- Probes to 4 -> finds empty
- Stops -> incorrectly thinks C is not present

Python's Solution: Tombstones

Instead of removing, Python marks the slot as “dummy” (deleted but not empty).

Index: 3 4 5

 [A] [X] [C]

- X = deleted marker (tombstone)
- Lookup continues past it
- Insertion can reuse it

Too many dummy entries cause:

- slower lookups.
- longer probe chains.
- Python handles this by resizing / rebuilding the table, removing dummy entries in the process.

```
def delete(table, key):  
  
    h = hash(key)  
  
    i = h % table.size  
  
    while table[i] is not empty:  
        if table[i].key == key:  
            table[i] = DUMMY  
            return  
        i = probe_next(i)  
  
    raise KeyError
```

Rebuilding the Hash Table

Load Factor (α) = number of stored elements / table size.

- If α is too low $\rightarrow$ memory wasted
- If α is too high $\rightarrow$ more collisions $\rightarrow$ slower operations

“Number of stored elements” includes:

- Actual elements.
- Deleted markers (tombstones), because tombstones still affect performance.

Normally rebuilding the table is initiated after insertion operation and the complexity of rebuilding is included with insertion's complexity. Since, the rebuilding happens rarely, thus the complexity is amortized $O(1)$.

Python keeps α around 0.66 (2/3) to balance:

- speed
- memory usage

Entry is valid if it is not a empty slot or deleted (dummy) slot.
Now the pseudo-code for resizing the table is,

```
def resize(old_table):  
  
    new_size = old_table.size * 2  
  
    new_table = create_table(new_size)  
  
    for entry in old_table:  
  
        if entry is valid:  
  
            insert(new_table, entry.key, entry.value)  
  
    return new_table
```